PERSONAL FINANCE ADVISOR BOT

Project Documentation

An AI-assisted application for income and expense tracking,
budget planning, spending analysis, and savings guidance.

Prepared for: Academic / Portfolio Project
Repository: https://github.com/ayushchavan0402-cmyk/Personal-Finance-advisor-bot

1. Project Overview

The Personal Finance Advisor Bot is an AI-powered personal financial planning application. It helps users organize income and expenses, review spending by category, generate budget suggestions, set savings goals, and view a summarized picture of their finances. The repository describes a Flask backend, SQLAlchemy with SQLite persistence, optional Gemini AI integration, optional Google Sheets synchronization, and a rule-based fallback for core workflows.

1.1 Problem Statement

People may find it difficult to track everyday spending, understand which categories consume the most money, plan a realistic budget, and consistently work toward savings goals. This project brings common tracking and analysis tasks into one interface and provides structured guidance based on the information entered.

1.2 Objectives

- Record monthly income and categorized expenses.
- Support daily, weekly, and monthly expense logs.
- Generate budget suggestions based on income, expenses, and goals.
- Highlight spending patterns and possible savings opportunities.
- Provide a monthly summary/report.
- Offer AI-generated guidance when the Gemini integration is configured.
- Keep core budget and analysis workflows usable through local rule-based fallback logic.

2. Main Features

Feature	Description
Income tracking	Enter the income amount used for planning.
Expense categorization	Organize expenses into categories such as rent, food, transport, dining, entertainment, and utilities.
Expense logs	Record expenses across daily, weekly, or monthly periods.
Budget generation	Create a suggested budget based on the submitted financial details.
Spending analysis	Review spending patterns and category-level totals.
Saving suggestions	Receive suggestions intended to help identify possible savings.
Goal-based planning	Include a savings goal, such as an emergency fund, in the analysis request.
Monthly reporting	Generate a summary/report of the submitted financial data.
AI integration	Use the configured Gemini model for AI-assisted responses.
Fallback mode	Use local rule-based logic if AI is unavailable or not configured.
Optional Google Sheets sync	Send data to a configured Apps Script web app.
Browser history	The README describes LocalStorage browser history.

3. Technology Stack

Technology	Role
Python	Main application language.
Flask	Web framework and HTTP routes.
SQLAlchemy	Database access and persistence layer.
SQLite	Local database storage.
Google GenAI SDK / Gemini	Optional AI-assisted analysis and guidance.
HTML, CSS, JavaScript	Browser interface and interactive result rendering.
Fetch API	Asynchronous communication between frontend and backend.
Google Apps Script / Google Sheets	Optional cloud tracking layer.
ngrok	Optional public tunnel for development/demo access.

4. System Architecture

The repository README describes the following high-level request flow:

```
User's Browser
    │
    │  Fetch / AJAX request
    ▼
Flask Application (app.py)
    ├── Input validation and prompt construction
    ├── SQLAlchemy ──► SQLite database
    ├── Gemini API (optional)
    ├── Google Apps Script / Sheets (optional)
    └── Rule-based fallback

Response returned to browser for display
```

The frontend sends financial inputs to Flask. The backend validates and processes the request, uses the configured AI integration where available, and can fall back to local rules. Results are returned to the browser for rendering. SQLite is the local persistence layer; Google Sheets is described as optional.

5. Repository Structure

```
Personal-Finance-advisor-bot/
├── app.py                          # Main Flask application
├── requirements.txt                # Python dependencies
├── .env.example                    # Example environment configuration
├── .gitignore                      # Files excluded from Git
├── run_public.py                   # Optional public-tunnel runner
├── google_apps_script.gs           # Optional Sheets integration template
├── index.html                      # HTML entry/interface file
├── README.md                       # Project overview and setup
├── QUICKSTART.md                   # Quick setup instructions
├── IMPLEMENTATION_SUMMARY.md       # Implementation notes
├── DEPLOYMENT_CHECKLIST.md         # Deployment checks
├── PROJECT_DELIVERY_SUMMARY.md     # Delivery summary
└── Documentation/                  # Additional documentation
```

The README also describes a templates/ directory containing index.html and a static/ directory for style.css and script.js. Confirm the current folder tree in the repository before relying on those paths, because the GitHub file listing may evolve.

6. Installation and Setup (Windows)

Prerequisites: Python 3.8 or later, Git (optional if downloading ZIP), a browser, and an editor such as VS Code.

1. Clone the repository or download it from GitHub and open the project folder in VS Code.
2. Open the VS Code terminal in the project directory.

```
git clone https://github.com/ayushchavan0402-cmyk/Personal-Finance-advisor-bot.git
cd Personal-Finance-advisor-bot
```

Create and activate a virtual environment:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

If PowerShell blocks activation, use Command Prompt and run:

```
venv\Scripts\activate.bat
```

Install the dependencies:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Create a local .env file using .env.example as a guide. Add only the configuration values you actually use. For Gemini features, configure a valid Gemini API key and a supported model name according to the current project code.

Start the local application:

```
python app.py
```

Open the local address shown by Flask; the repository README lists http://127.0.0.1:5000 as the expected local URL.

7. Configuration

Setting	Purpose / caution
Gemini API key	Used for AI-assisted analysis when configured. Keep it private and do not commit it to GitHub.
GEMINI_MODEL	Configurable model name. The repository README says it defaults to gemini-2.5-flash; check current code and provider availability.
GOOGLE_SHEETS_WEBHOOK_URL	Optional Apps Script web-app URL for Google Sheets sync.
NGROK_AUTHTOKEN	Optional token used by the public-tunnel runner.

Never publish .env, API keys, authentication tokens, real personal financial records, or other secrets.

8. Application Routes and API Overview

Route	Purpose

GET /	Serves the home page / application interface.
GET /api/health	Returns application, database, and AI configuration status.
POST /analyse	Accepts income, goal, expenses, and optional expense logs; returns budget/analysis/suggestions/summary information.
POST /generate	Compatibility endpoint for focused financial questions; the README says it accepts question or product_info.

8.1 Example Analysis Request

```
{
  "income": 50000,
  "goal": "emergency fund",
  "expenses": {
    "rent": 12000,
    "food": 6000,
    "transport": 3000,
    "dining": 2000,
    "entertainment": 1500,
    "utilities": 3000,
    "savings": 5000
  },
  "expense_logs": []
}
```

Illustrative example only. The values are not real user financial data. The README describes a successful response as including fields such as success, source, budget, analysis, suggestions, summary, report_id, and cloud_sync.

9. Typical User Workflow

3. Open the application in a browser.
4. Enter income, expense amounts/categories, and a savings goal if applicable.
5. Submit the form for analysis.
6. Review the suggested budget, spending analysis, savings suggestions, and summary.
7. Use the results as a planning aid and adjust entries to reflect actual circumstances.
8. If configured, review optional cloud-sync behavior and ensure the destination is appropriate for sensitive data.

10. Testing Checklist

- ☐ Application starts without import or configuration errors.
- ☐ Home page loads in the browser.
- ☐ Health endpoint responds and reports service status.
- ☐ Valid income and expense inputs return a structured analysis.
- ☐ Missing, malformed, negative, or unusually large values are handled safely.
- ☐ Core budget workflow remains available when Gemini is not configured, if fallback behavior is intended.
- ☐ Database records persist as expected between application restarts.
- ☐ Optional Google Sheets integration handles success and failure cases.
- ☐ UI remains usable on desktop and mobile screen sizes.

- ☐ No secrets or private financial data are committed to the repository.

11. Deployment Notes

The repository includes a deployment checklist and an optional ngrok runner. For a demonstration, local Flask execution is the simplest starting point. Before exposing the application publicly, review debug settings, secret management, input validation, database access, rate limits, HTTPS, and whether any submitted data is stored or synced. Do not treat a development tunnel as a production hosting solution.

The repository currently links to a hosted frontend/demo URL in its GitHub About section. The live deployment may differ from the local code, so verify the current deployed version and backend configuration before presenting it.

12. Limitations and Future Enhancements

Potential limitations to evaluate during testing:

- AI-generated output may be incorrect or overly general.
- The quality of analysis depends on accurate and complete user-entered data.
- AI and cloud synchronization depend on valid external configuration and network availability.
- Authentication, authorization, and multi-user isolation should be verified before storing real financial data.
- Financial data needs careful privacy, retention, and access controls.

Future enhancements listed or suggested by the project documentation include:

- Predictive spending analytics and month-to-month trend charts.
- Goal-based savings progress tracking.
- Household or shared budgets.
- Authentication and stronger account-level privacy controls.
- Multi-month financial health monitoring.
- Exportable reports and improved data visualization.

13. Security, Privacy, and Financial Disclaimer

This application is an educational financial-planning aid. It is not a substitute for a qualified financial professional and does not guarantee outcomes or provide personalized regulated investment advice. Users should independently verify important decisions.

Security recommendations: keep API keys in environment variables; exclude .env and local database files from Git; avoid using real bank credentials; validate and sanitize user input; protect stored data; use HTTPS for public access; and review third-party AI/cloud data handling before sending sensitive information.

14. Conclusion

The Personal Finance Advisor Bot brings together income and expense tracking, budget generation, spending analysis, savings suggestions, and monthly summaries in a single application. Its Flask backend, SQLite persistence, optional Gemini integration, and rule-based fallback provide a foundation for an academic and portfolio project. Further testing, privacy safeguards, and clear documentation can strengthen its reliability and presentation.

15. Project Links

GitHub repository: https://github.com/ayushchavan0402-cmyk/Personal-Finance-advisor-bot

Demo link listed by repository: https://personal-finance-advisor-bot-tau.vercel.app/

Documentation basis: the public GitHub repository and its README were reviewed to prepare this document. Descriptions are based on the repository documentation and should be checked against the current implementation before formal submission.